scan_engine.py

Method-by-method reference

ThorlabsControl

Overview

scan_engine.py is the headless core of the scanning system: it owns the raster-scan acquisition loop, the motion-profile-based math that turns a burst of time-stamped ADC samples into X positions, the resampling/caching used to turn a scan into an image, and basic diagnostics. It has no GUI code of its own -- scan_gui.py imports run_scan(), load_scan(), build_scan_grid_incremental(), build_scan_lines_incremental(), and check_connection_and_home() from this module and wraps them in a Tkinter front end.

The module's functions fall into four groups, in the order they appear in the file:

- Geometry / I-O: build_axis_points() plans the Y positions to visit; load_scan() reloads a previously saved scan.
- Position reconstruction: the private _row_positions()/_cached_row_positions() helpers turn one row's raw ADC samples into X positions using the modeled trapezoidal motion profile, anchored by a measured move-start lag.
- Visualization data prep: build_scan_grid() / build_scan_lines() and their *_incremental() cached counterparts resample rows for a heatmap, 3D surface, or overlaid-lines view; plot_scan() is the standalone matplotlib viewer.
- Acquisition: check_connection_and_home() (pre-flight homing), print_scan_timing_stats() (post-scan diagnostics), and run_scan() itself, which drives the motors and the Arduino through one full raster scan.

Data flows one way through these groups: run_scan() (or load_scan() on a saved file) produces scan_rows -- a plain list of dicts, one per scanned row, holding the raw ADC samples plus every timing/lag measurement taken while acquiring that row. Everything in the visualization group is a pure function of scan_rows; nothing downstream of acquisition talks to hardware.

A recurring theme in this file, worth keeping in mind while reading the reference below: the X position of an ADC sample is never measured directly. It is reconstructed after the fact from an idealized trapezoidal accel/cruise/decel model of the move (motion_timing.sample_positions_for_motion), anchored per row by a measured move-start lag (how long after the move command was issued the stage actually started moving). The run_scan() reference section below explains how that lag -- and, more recently, a measured motion-stop instant -- is obtained via MoveStartLagMonitor and used to make sure a row's burst read never ends before the row's motion actually does.

Module-level constant

COVERAGE_WARN_THRESHOLD = 0.97

Threshold used at the end of each row inside run_scan(). After a row is acquired, its samples are re-mapped to X positions the same way the plots do, and the covered span is compared against the commanded x_span. If less than 97% of the commanded span was actually captured, a per-row warning is printed -- a sanity check on the stop-detection machinery described under run_scan(): if the read length were ever silently truncated before the stage finished moving, this is what would catch it.

Geometry and file I/O

build_axis_points(start: float, span: float, spacing: float)

```
build_axis_points(start, span, spacing) -> list[float]
```

Builds the evenly spaced list of points along one axis that a scan should visit -- used for the Y axis in run_scan() (one entry per scanned row), and equally applicable to X.

Parameters

- start -- coordinate of the first point.
- span -- signed total distance to cover; its sign sets the scan direction. span == 0 returns a single point ([start]), i.e. a one-line scan.
- spacing -- distance between consecutive points; must be > 0 or the function raises ValueError.

Returns

A list of floats starting at start and stepping by spacing in the direction of span. The true endpoint (start + span) is always included even if it doesn't land exactly on a spacing multiple -- it is appended as an extra final point if needed -- so the full requested span is always covered, never truncated to the last even multiple of spacing.

---

load_scan(path)

```
load_scan(path) -> list[dict]
```

Reloads a scan_rows list previously written to disk by run_scan()'s save_file argument (an np.savez() dump). Uses allow_pickle=True since each row is a dict containing nested arrays, lists, and None values, not a flat numeric array. This is what lets scan_gui.py's "Load Saved Scan..." button, and plot_scan(), work on old data without touching hardware.

Position reconstruction (private helpers)

These two helpers are the shared core that every visualization function below builds on. Both are prefixed with an underscore -- internal to this module, not part of its public interface.

_row_positions(row) (private helper)

```
_row_positions(row: dict) -> numpy.ndarray
```

Converts one scan_rows row into an array of X positions, one per raw ADC sample, by calling motion_timing.sample_positions_for_motion() with that row's x_start, x_displacement, sample count, actual captured sample_span_s, and the acceleration/max_velocity that were in effect for that row. Critically, it passes motion_start_offset_s -- the measured move-start lag captured live during acquisition -- as the profile's time origin, instead of assuming the stage started moving exactly when the move command was issued. This is what keeps the modeled trapezoid lined up with the real move despite command/response latency.

_cached_row_positions(row, positions_cache) (private helper)

```
_cached_row_positions(row: dict, positions_cache: dict) -> numpy.ndarray
```

Memoizing wrapper around _row_positions(), keyed by id(row) -- the row dict's Python object identity. Safe because scan_rows only ever grows by appending new row dicts; existing rows are never mutated in place, so a cached result never goes stale. Used by the *_incremental() functions below so a GUI redrawing every ~250 ms during a live scan doesn't recompute the (non-trivial) position reconstruction for rows it already saw on a previous redraw.

Visualization data preparation

build_scan_grid(scan_rows, max_points=200)

```
build_scan_grid(scan_rows, max_points=200)
    -> (x_grid: ndarray, y_grid: ndarray, z: ndarray)
```

Resamples the ragged set of per-row samples (rows differ in X start/end -- direction alternates row to row -- and in sample count) onto one common rectangular grid, so the scan can be drawn as a heatmap or 3D surface.

Algorithm

- Drop empty rows (no samples collected), then sort the remaining rows by Y.
- Build x_grid as n_cols evenly spaced points spanning the min/max X across all rows, where n_cols = min(max_points, the largest per-row sample count).
- For each row: reconstruct its true X positions via _row_positions(), then interpolate its samples onto x_grid with motion_timing.interp_nan(), which leaves grid cells outside that row's actually-sampled X range as NaN rather than extrapolating.
- If there are more rows than max_points, downsample rows too, by picking max_points evenly spaced row indices (point selection, not averaging).

Returns

x_grid, y_grid (1D coordinate arrays) and z with shape (len(y_grid), len(x_grid)); NaN marks a cell that fell outside what was actually sampled. This function always recomputes from scratch -- an O(rows x columns) cost on every call -- see build_scan_grid_incremental() below for a cached version meant for repeated redraws.

build_scan_lines(scan_rows)

```
build_scan_lines(scan_rows) -> list[(y, x_positions: ndarray, samples: list)]
```

A simpler alternative to a resampled grid: returns each non-empty row's own real (unresampled) X positions and samples as an independent line, for an "overlaid lines" view where every row is drawn as its own trace rather than fit onto a shared grid -- no interpolation or downsampling error is introduced. Rows are returned in scan_rows order (not sorted by Y, unlike build_scan_grid()).

build_scan_grid_incremental(scan_rows, cache, max_points=200)

```
build_scan_grid_incremental(scan_rows, cache: dict, max_points=200)
    -> (x_grid: ndarray, y_grid: ndarray, z: ndarray)
```

Cached equivalent of build_scan_grid(), for callers -- specifically scan_gui.py's redraw loop -- that call it repeatedly while scan_rows keeps growing during a live scan. The caller owns the cache dict across calls; pass a fresh {} to start over (a new scan, or a freshly loaded file).

Why it matters

Without caching, every redraw during a long scan would repeat the position reconstruction and grid interpolation for every row seen so far, not just the new ones -- work that grows every tick as the scan progresses. That is wall-clock time spent holding the GIL on whichever thread calls this function; if called carelessly from a GUI's main thread it can starve a concurrently running hardware-read thread. (scan_gui.py avoids this by running the render itself on a dedicated background thread -- see its _service_render().)

Cache structure

- cache["positions"]: id(row) -> reconstructed X-position array, shared with build_scan_lines_incremental() below -- position reconstruction doesn't depend on the grid, so it survives a grid rebuild.
- cache["z_rows"]: id(row) -> that row's samples already interpolated onto the current grid, keyed additionally by cache["grid_key"] = (x_min, x_max, n_cols). If the grid geometry changes (e.g. the user edits max_grid_points, or a new scan has a different commanded span), grid_key changes, z_rows is invalidated and rebuilt, but the underlying position cache is untouched.

With a matching cache, a redraw where nothing new has arrived degenerates into a cheap dict-lookup pass rather than a full recompute.

---

build_scan_lines_incremental(scan_rows, cache)

```
build_scan_lines_incremental(scan_rows, cache: dict) -> list[(y, x_positions, samples)]
```

Cached equivalent of build_scan_lines(). If passed the same cache dict as build_scan_grid_incremental(), it reuses that dict's "positions" entries, so toggling a GUI between "Overlaid Lines" and "Heatmap"/"3D Surface" views doesn't reconstruct the same rows' X positions twice.

plot_scan(scan_rows)

```
plot_scan(scan_rows) -> None
```

The standalone, non-GUI visualization entry point: builds one line per row via build_scan_lines(), plots them all on a single matplotlib Axes (X position vs. ADC count), colored by row index through a viridis colormap, with a legend if there are 20 rows or fewer, then calls plt.show(). This is what the __main__ block at the bottom of the file calls after run_scan() finishes, and the usual way to inspect a file loaded via load_scan() from a plain script or REPL. Prints a message and returns without plotting if every row is empty.

Acquisition

check_connection_and_home(motorx, motory, home_timeout_s: float = 30.0)

```
check_connection_and_home(motorx, motory, home_timeout_s=30.0) -> None
```

Pre-flight hardware check and homing routine, run once before a scan when skip_homing_check is False (and directly by scan_gui.py's "Home Motors" button, independent of a full scan). For each axis in turn (X then Y):

- Requests a status update and prints the status bits, reported position, and the controller's actual polling interval -- a sanity check that the DLL connection is alive and configured as expected.
- Issues a blocking home cycle (home(wait=True, timeout_s=home_timeout_s)).
- Re-checks status bits and verifies the "homed" bit (0x00000400) is set, raising ThorlabsError if not -- so a silently-failed home doesn't let a scan proceed from an unknown reference position.

print_scan_timing_stats(scan_rows)

```
print_scan_timing_stats(scan_rows) -> None
```

Post-scan diagnostic summary, printed to stdout at the end of run_scan(). For each of the timing fields collected per row during acquisition it prints the avg/min/max in milliseconds across all rows: the delay from issuing the move command to the burst-read loop actually starting; the move_relative() call's own duration; the motion_start_offset_s used to anchor the X-position model; the actual captured sample span; the measured move-start lag's lower/upper bounds and midpoint; the moving-bit lag; and the extra read time spent past the nominal read_duration waiting on the stop_condition (see run_scan() below) -- a near-zero average here means rows are usually finishing within their nominal budget, a large one means the stop-condition extension is frequently doing real work. It also reports how many rows (out of the total) never detected a move-start lag, or never detected a confirmed motion-stop, within their monitor's timeout -- those rows silently fell back to less precise timing, worth knowing about even though the scan still completed.

run_scan(...)

```
run_scan(
    *, serial, arduino_port, arduino_baud,
    x0, y0, x_span, y_span, line_spacing,
    acceleration, max_velocity,
    row_settle_s=2.0,
    read_safety_factor=1.10, read_overhead_s=0.05,
    home_timeout_s=30.0, skip_homing_check=True,
    move_start_lag_threshold_mm=0.0000005,
    lag_monitor_timeout_s=2.0,
    save_file=None,
    progress_callback=None,
    stop_event=None,
) -> list[dict]
```

The core of the module: drives one full raster scan (a boustrophedon/serpentine sweep in X, stepped in Y) and returns the resulting scan_rows.

Parameters

- serial, arduino_port, arduino_baud -- hardware addressing: the Kinesis controller's serial number and the Arduino's COM port/baud rate.
- x0, y0, x_span, y_span, line_spacing -- scan geometry. The scan covers [x0 - x_span/2, x0 + x_span/2] by [y0 - y_span/2, y0 + y_span/2]; rows are spaced line_spacing apart along Y via build_axis_points().
- acceleration, max_velocity -- motion-profile parameters, applied to both axes via set_velocity_params().
- row_settle_s -- dwell time after each Y move, before the next row's X sweep starts, to let residual vibration settle.
- read_safety_factor, read_overhead_s -- pad the nominal read duration (sampling_duration_for_move) beyond the modeled move time, as a first line of defense before the stop_condition mechanism (see below) becomes the deciding factor.
- home_timeout_s, skip_homing_check -- whether/how long to run check_connection_and_home() before scanning.
- move_start_lag_threshold_mm, lag_monitor_timeout_s -- tuning passed through to MoveStartLagMonitor (see motion_timing.py) for detecting real motion start/stop.
- save_file -- optional path; if given (and any rows were collected), scan_rows is written there via np.savez() for later reloading with load_scan().
- progress_callback(row_dict, row_index, total_rows) -- optional hook invoked after each row is appended, e.g. so a GUI can update a live progress display without waiting for the whole scan.
- stop_event -- optional threading.Event, checked before each new row starts; if set, the scan ends early with whatever rows are already complete, motors left stationary at the end of the last finished row rather than mid-move.

Setup

Computes x_start/y_start and the list of y_points, then -- inside a try/finally that guarantees hardware teardown -- connects both axis controllers (X on channel 1, Y on channel 2, both polling their

position/status cache at 1 kHz), optionally runs check_connection_and_home(), pushes acceleration/max_velocity to both axes, and moves both to the scan's starting corner (blocking). It then opens the Arduino connection and computes move_time and read_duration for one X sweep -- shared by every row, since the sweep geometry doesn't change row to row.

Per-row loop

For each y_target in y_points:

- Check stop_event; break out of the loop if it's set.
- For every row after the first: move Y to y_target (blocking) and sleep row_settle_s.
- Alternate the X sweep direction every other row (row_index % 2) -- a serpentine raster that avoids an extra fly-back move per row.
- Reset the Arduino's input buffer and record the queried (not commanded) actual_x_start / actual_y, plus timestamps for later analysis.
- Start a MoveStartLagMonitor(watch_for_stop=True) before issuing the move. Running in its own background thread, it both detects the real move-start lag (a position-threshold crossing) and watches the moving status bit for a confirmed start-then-stop transition, exposed via has_stopped(). Its own timeout now covers the whole row, not just the start-lag window, since it has to stay alive for the stop check too.
- Issue the X move non-blocking (move_relative(..., wait=False)).
- Call burst_read_binary() with read_duration as a minimum rather than a fixed length: stop_condition=lag_monitor.has_stopped and hard_timeout=row_wait_timeout_s. Once the nominal duration elapses, the read keeps draining the serial port and re-polling has_stopped() every few milliseconds until the monitor confirms the row's motion has actually finished (or the hard timeout is hit as a last-resort backstop) -- this is what stops a row's trailing samples from being cut off when a real move runs longer than the model predicted.
- Call motorx.wait_until_stopped() as an independent confirmation the move has ended, then read actual_x_end.
- Only now stop() and join() the lag monitor -- safe because the move is confirmed over -- and pull its result(): the lag bounds/midpoint, moving-bit lag, and motion-stopped lag. If no lag was ever detected within the timeout, fall back to command-return timing and print a warning.
- Append a row dict to scan_rows holding the row index, actual Y, actual X start/end, the commanded displacement, motion_time and read_duration, the row's actual captured sample span and its start timestamp, motion_start_offset_s (the anchor used by _row_positions()), several raw timing deltas, the lag monitor's measurements, the acceleration/max_velocity in effect, and the raw ADC samples.
- Print a one-line summary for the row, then reconstruct its X positions the same way the plots do and compare the captured span against x_span -- if it covers less than COVERAGE_WARN_THRESHOLD, print a warning (see that constant, above).
- Call progress_callback if one was given.

Teardown

After the loop (or an early stop_event break), print_scan_timing_stats() runs and scan_failed is set to False -- only reached if nothing raised. The finally block always closes the Arduino connection; if the scan didn't complete cleanly (scan_failed still True), it calls the more defensive safe_shutdown() on both motors, otherwise a normal disconnect(). Finally, if save_file was given and rows were collected, scan_rows is written to disk, and the function returns scan_rows.

if __name__ == "__main__": ...

The module's example/default standalone entry point. Running scan_engine.py directly performs one scan with hard-coded example parameters (serial 50865380, Arduino on COM3 @ 230400 baud, a 1 mm X span and 0.2 mm Y span centered at (1, 1), 0.1 mm line spacing, 4 mm/s^2 acceleration, 2 mm/s max velocity), saves the result to scan_last.npz, and calls plot_scan() to display it. This mirrors what scan_gui.py's "Launch Scan" button does, minus the GUI's live-progress and cancellation plumbing.